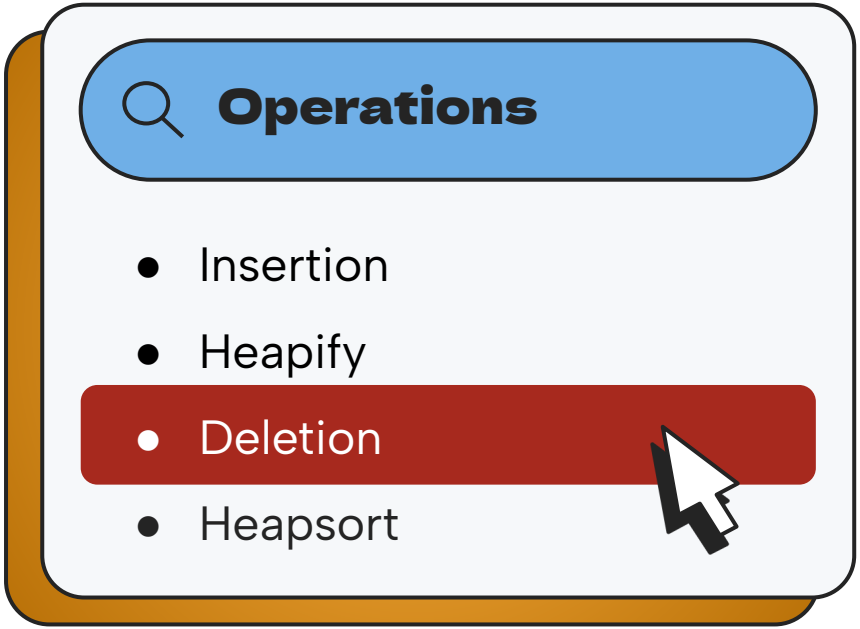


Data Structure(CSL202)

Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)

Operations in Heap



Deletion from a Maxheap

```
DeleteMax(Heap T)

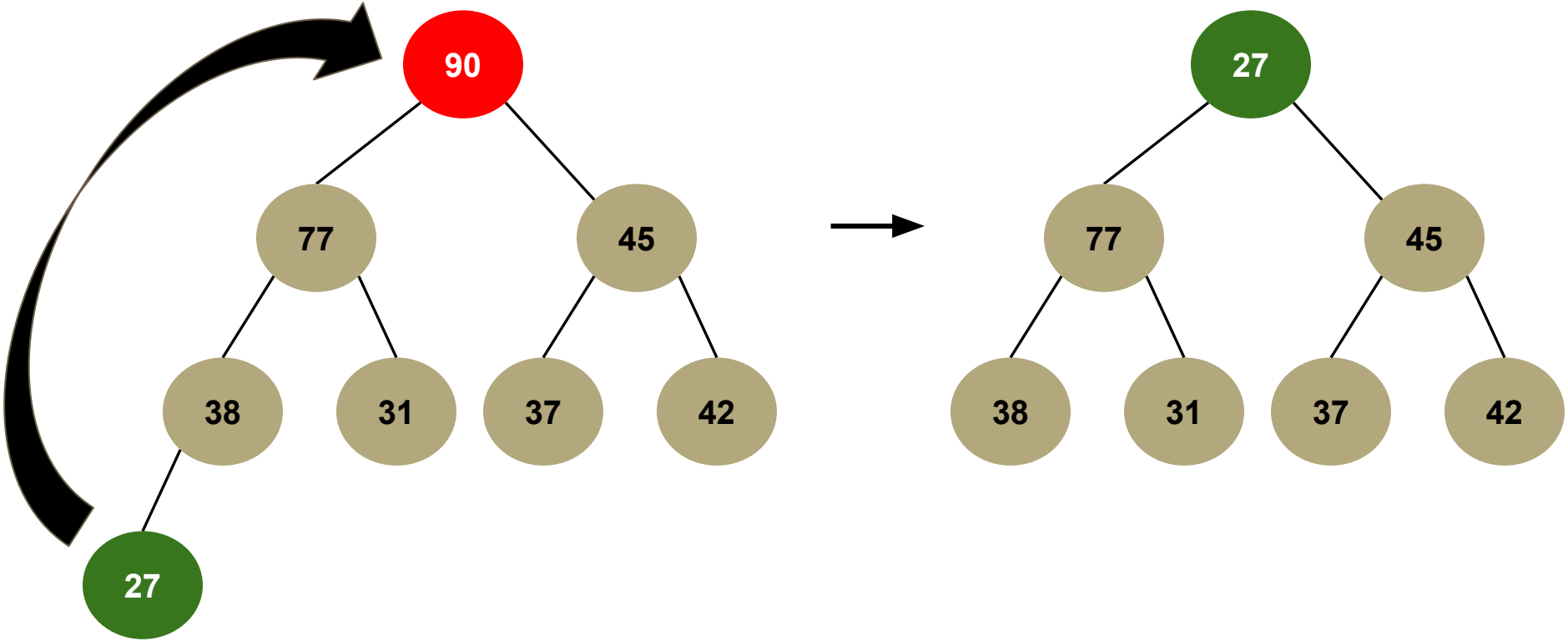
1  Swap(T, 0, T.size − 1);        // Move max (root) to the end

2  T.size := T.size − 1;          // Remove it from the heap

3  Heapify(T, 0);                 // Restore heap property from root down

4  return T[T.size];              // Return the removed max value
```

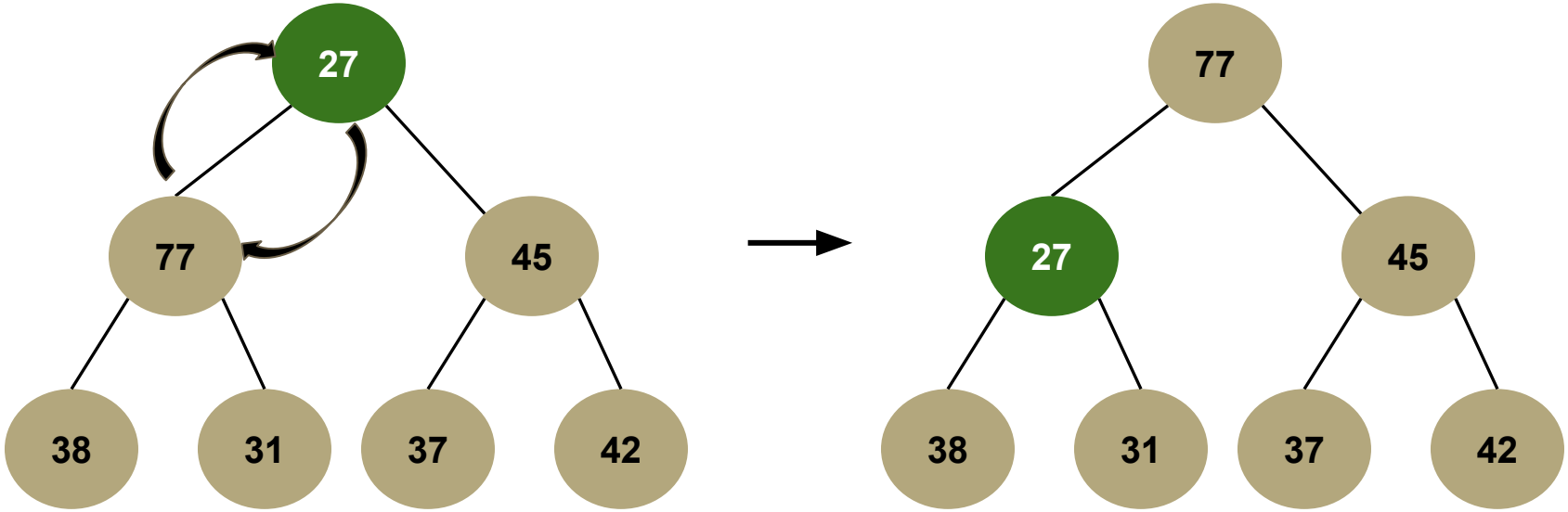
Deletion from a Maxheap



a. Delete 90

b.

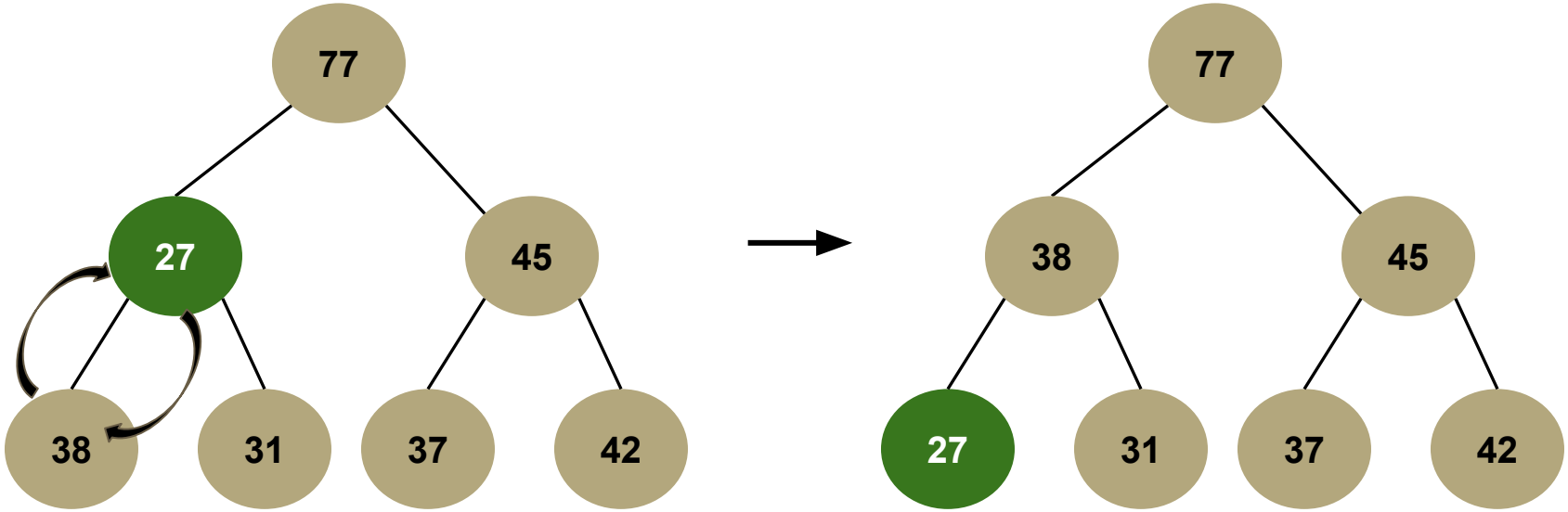
Deletion from a Maxheap



b.

c.

Deletion from a Maxheap

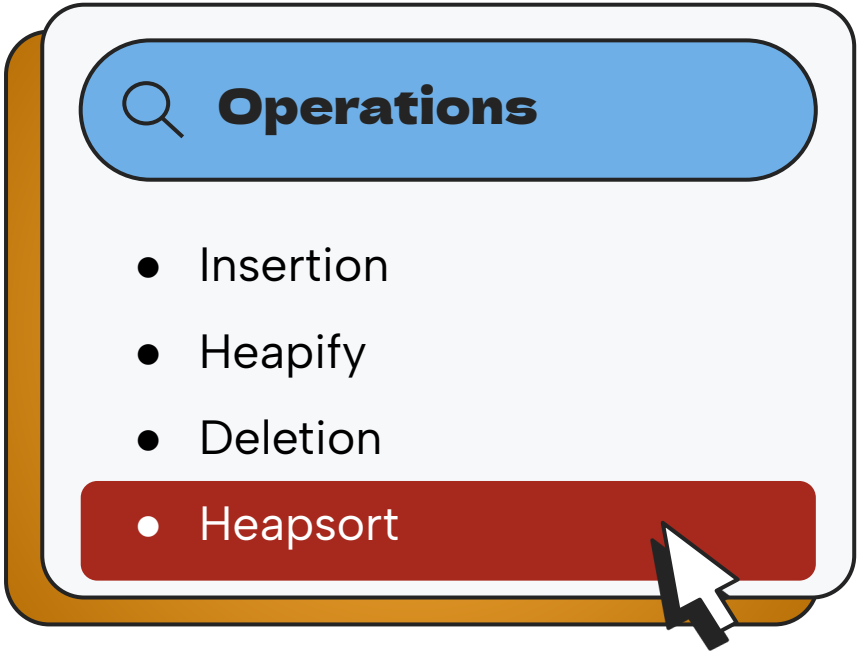


c. d.

Deletion from a Maxheap Complexity:

- **Best Case: O(1) –** No swaps needed.
- **Average Case: O(log n) –** Swaps halfway up.
- **Worst Case: O(log n) –** Swaps up to root.

Operations in Heap



Heapsort

```
HeapSort(Tree T)

1  T.size := |nodes of T|          // Initialize heap size to number of elements

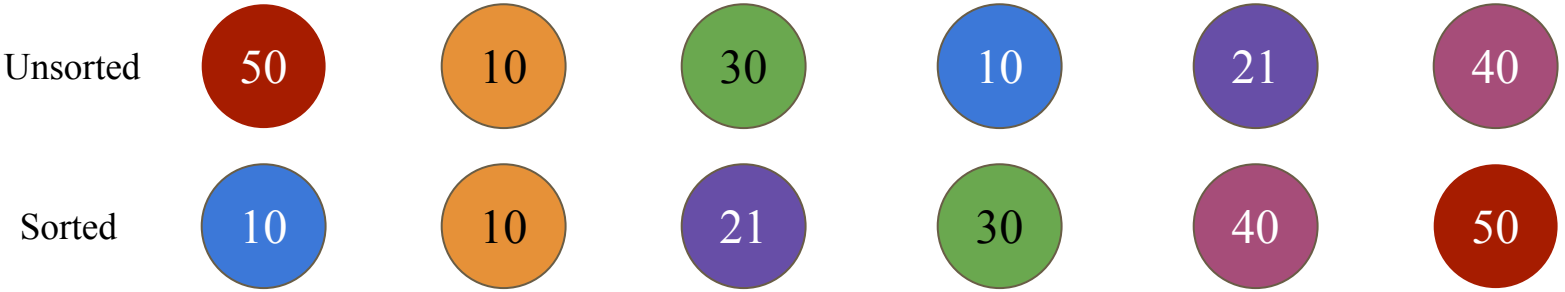
2  BuildHeap(T)                    // Convert the array into a valid maxheap

3  while T.size > 0 do             // Repeat until heap is empty

4      DeleteMax(T)                // Remove the max element and place it at the end
```

Heapsort

- **Stable Sort:** No, Heapsort may swap equal elements, which can change their order.



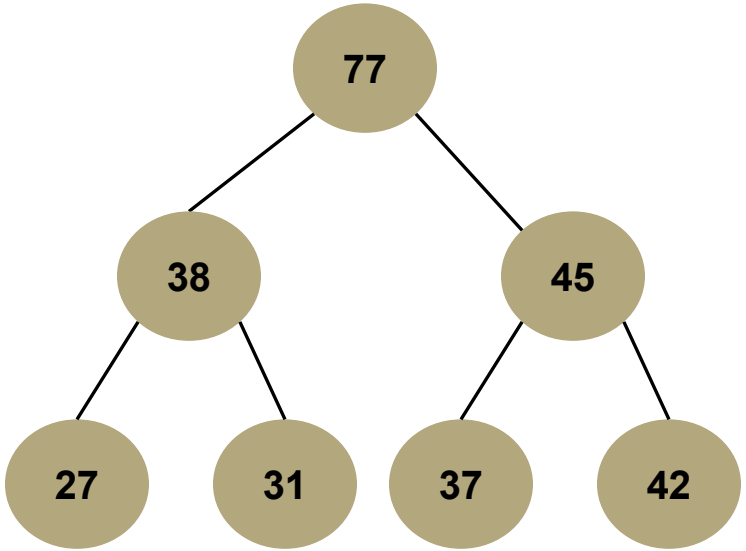
- **In-place:**
  - Yes.
  - O(1).

Heapsort

Unsorted array

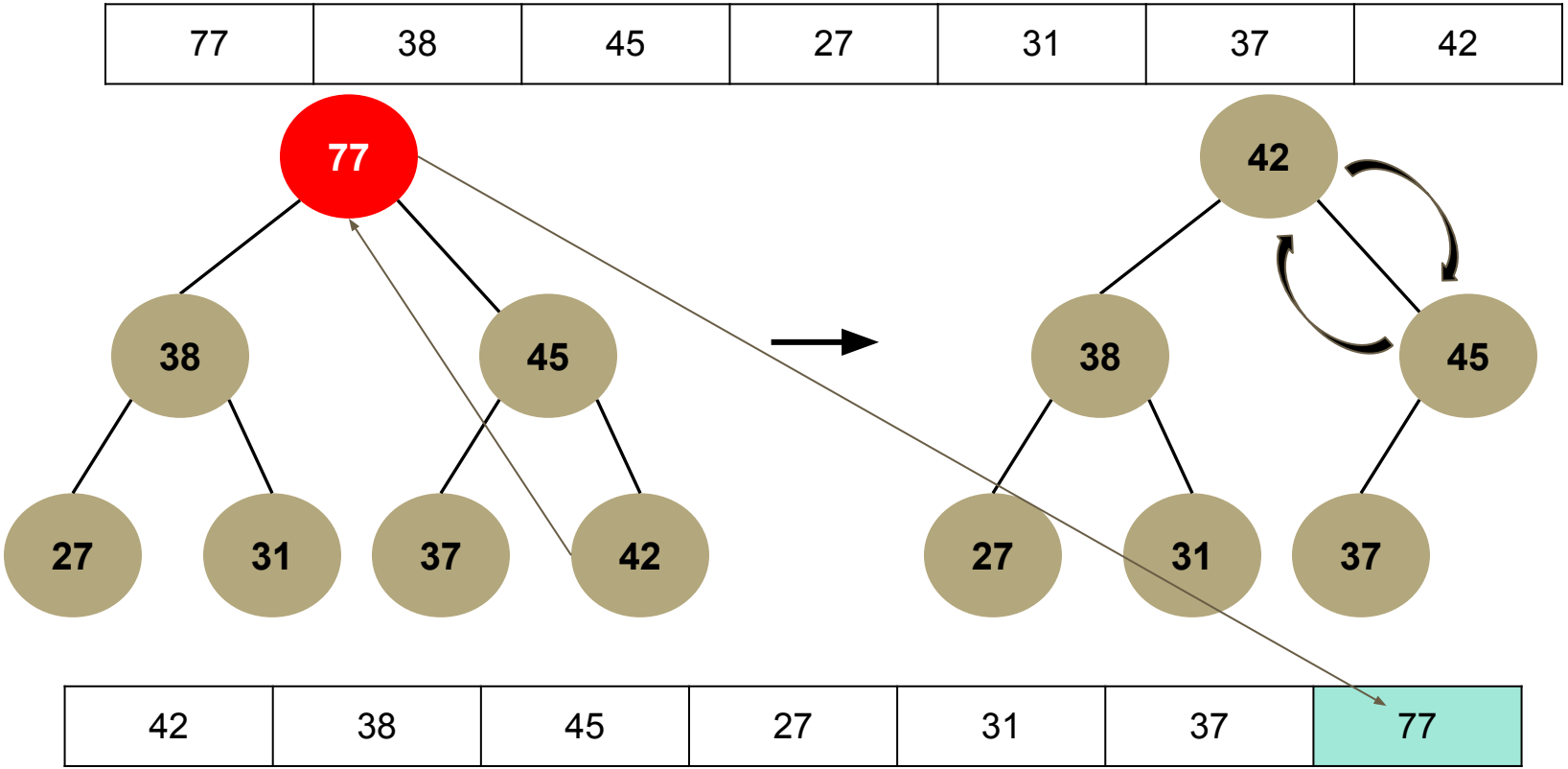
77	38	45	27	31	37	42

Build a maxheap



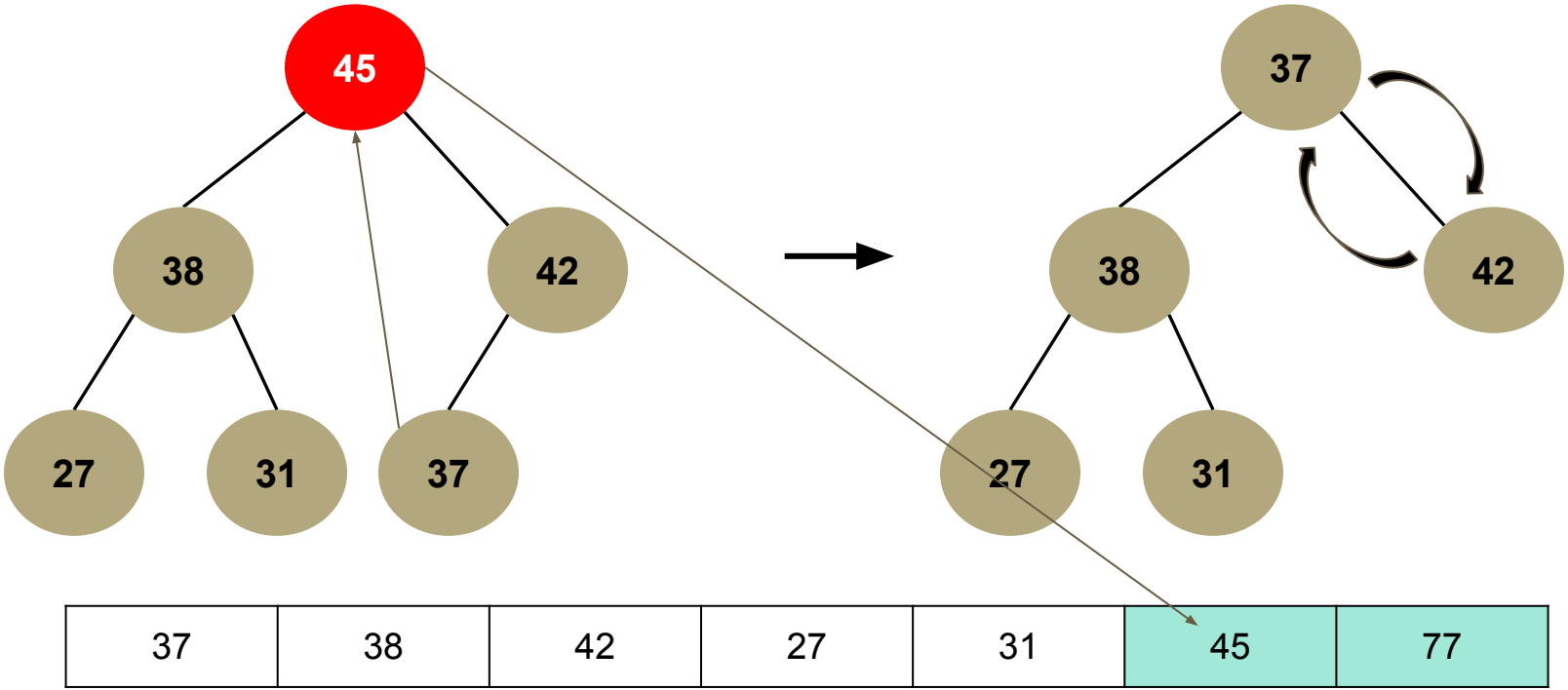
Heapsort

Delete 77



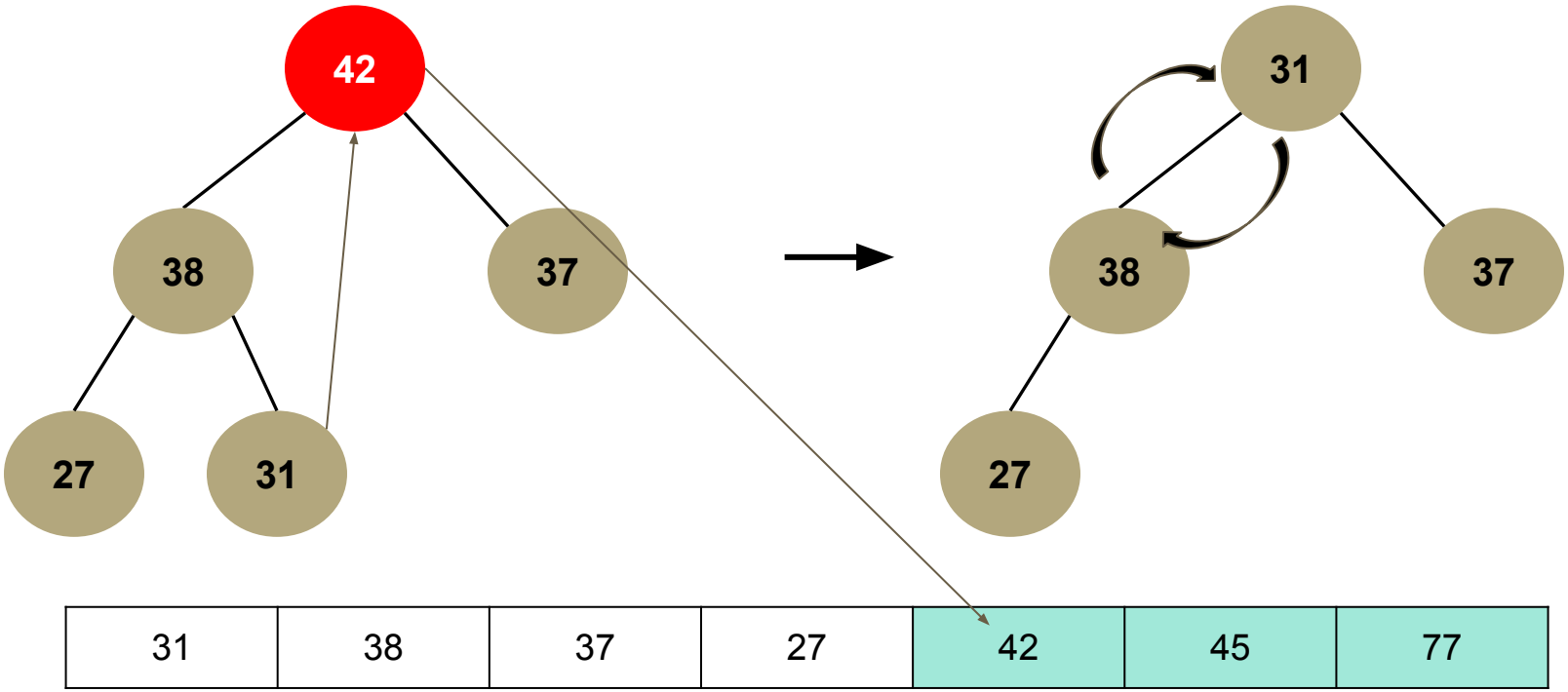
Heapsort

Delete 45



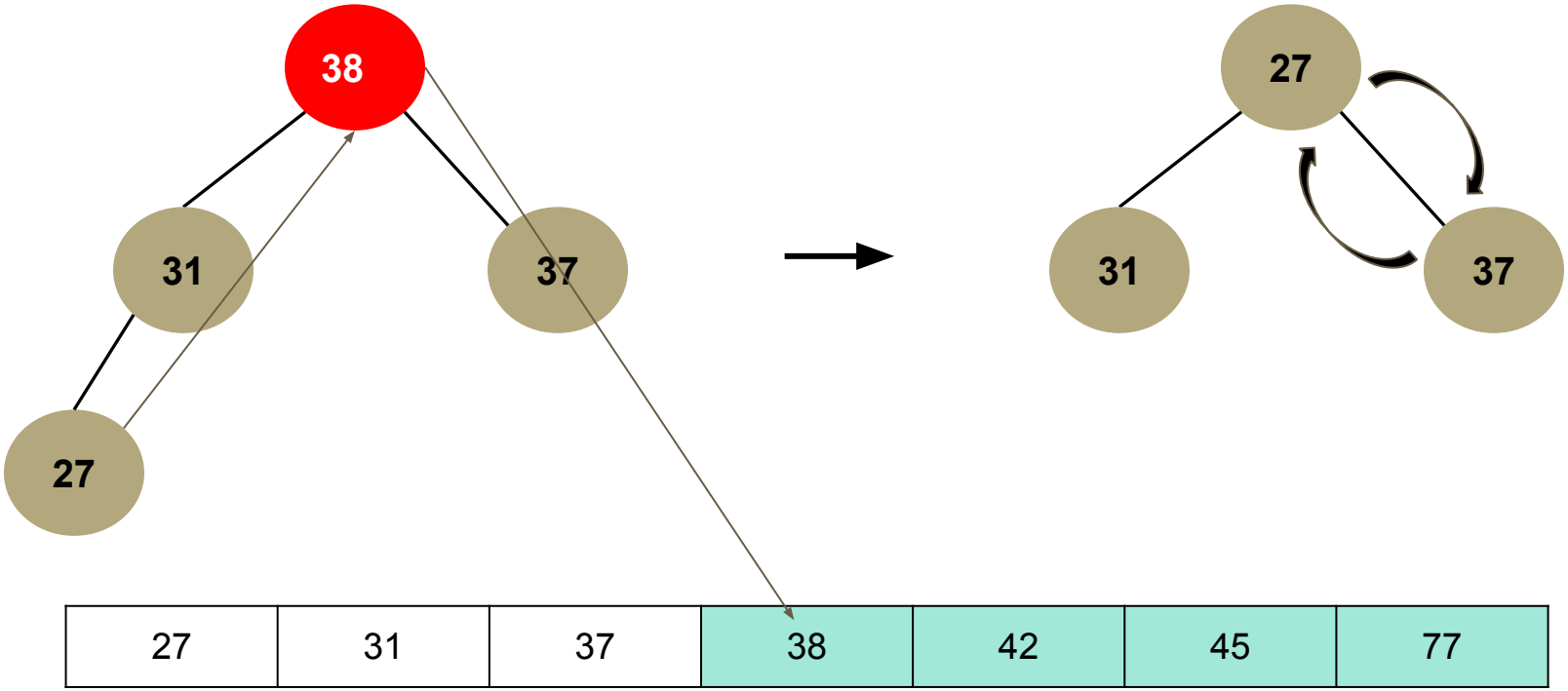
Heapsort

Delete 42



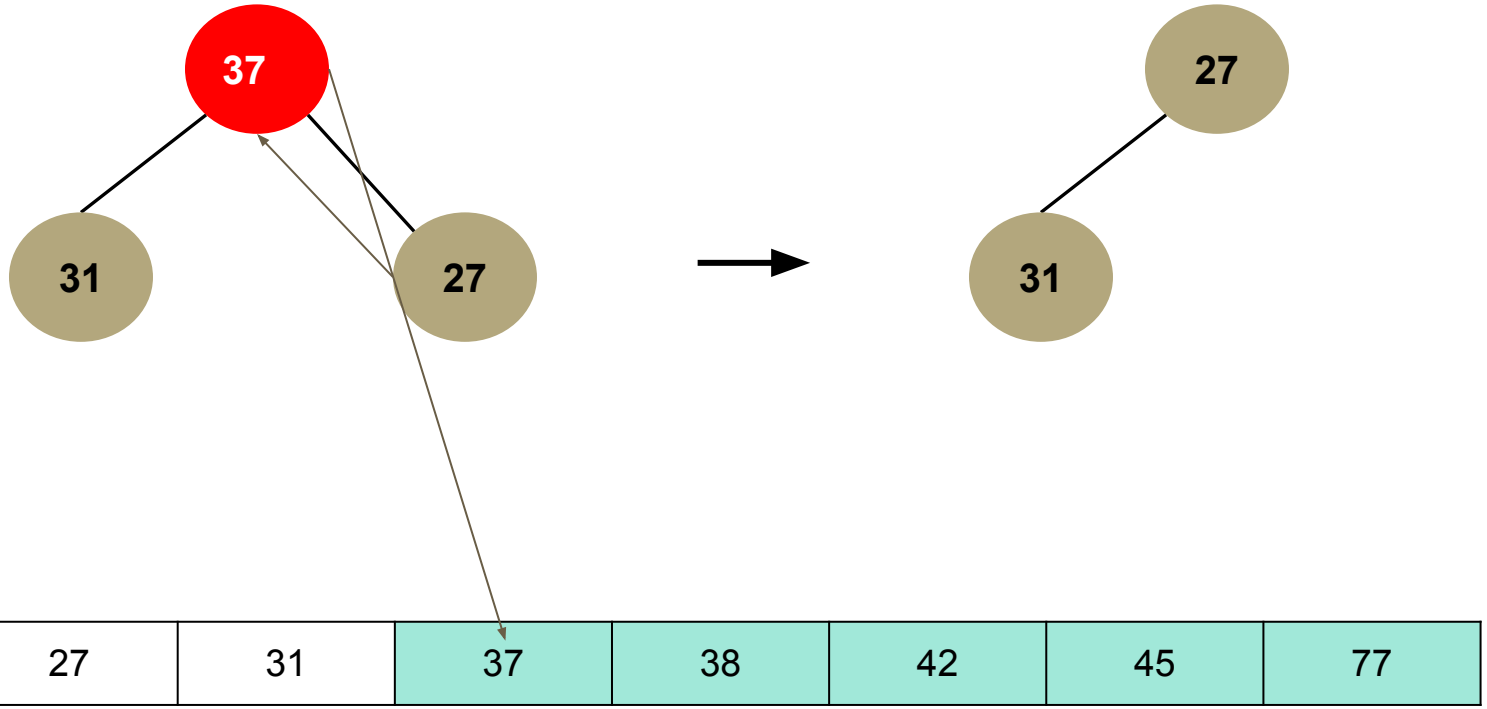
Heapsort

Delete 38



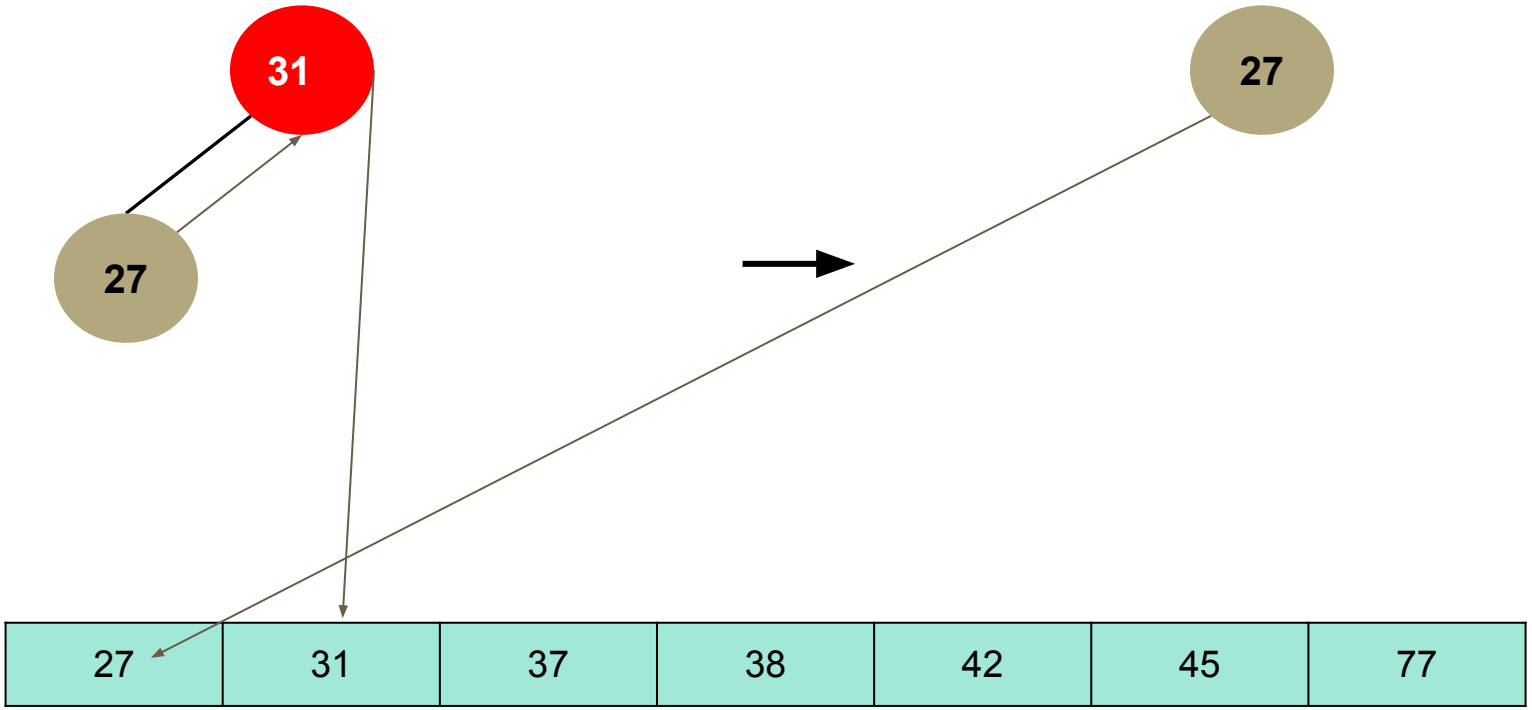
Heapsort

Delete 37



Heapsort

Delete 31



Heapsort

Array Sorted in Ascending Order

27	31	37	38	42	45	77

Heapsort Complexity:

- **Best Case:** O(n log n) – already sorted input.
- **Average Case:** O(n log n) – normal random input.
- **Worst Case:** O(n log n) – reverse sorted input.